

Building a Fine-Tuning Dataset

The model learns exactly what you show it, including the mistakes.



Should You Fine-Tune At All

Most problems people fine-tune for are not fine-tuning problems

Need the model to **know new facts**

USE RAG

Need a **consistent output format or tone** that prompting keeps missing

FINE-TUNE

Need to **shorten a long system prompt** used on every request

FINE-TUNE

Need a **smaller model** to match a larger one on one narrow task

BEST ROI

Have you exhausted **prompting and retrieval** first

DO THAT

🔧 THE HONEST TEST

If you cannot write a prompt that produces the behaviour **even once**, fine-tuning will probably not teach it either.

“ Fine-tuning makes existing capability **reliable**. It rarely creates capability that was never there.



AI Coach John

Follow



🔄 Repost

Behaviour, Not Knowledge

What the weights absorb well, and what they never will

✓ LEARNS WELL

output format and structure
tone, register, persona
domain phrasing and jargon
task specific reasoning patterns
when to refuse or defer
consistent JSON schemas
tool calling conventions

✗ LEARNS POORLY

new factual knowledge
information that changes often
anything needing a citation
long tail rare facts
things it could look up instead

Everything here belongs in RAG

🧠 A USEFUL MENTAL MODEL

prompting = instructions before each task
RAG = reference material before each task
fine-tuning = changing the default behaviour, so neither
is needed every time



**The dataset is a set of demonstrations of the behaviour you want as default.
That framing decides everything else.**



JSONL, One Example Per Line

Three roles, and one rule that governs all of them

JSONL

```
{ "messages": [  
  { "role": "system", "content": "You are a support triage  
    assistant. Reply in the required JSON schema." },  
  { "role": "user", "content": "My card was charged twice for  
    order #4471." },  
  { "role": "assistant", "content": "{ \"category\": \"billing\",  
    \"severity\": \"high\", \"order_id\": \"4471\" }" }  
]}
```

SYSTEM

The standing instruction. If production sends one, it must appear here identically.

USER

The input, and it should look like real user input: typos, fragments and all.

ASSISTANT

The target. Exactly what you want in production, never "close enough".



Providers differ, the principle does not: **training structure must match inference structure exactly**. Any mismatch costs quality.



Train It In The Context It Will See

The single most common structural mistake

✗ MISMATCHED

TRAINING

system: "You are a triage bot."

PRODUCTION

system: "You are a triage bot.

Today is {date}. JSON only.

Be concise."

Trained in a context it never sees again

✓ MATCHED

Build training examples FROM the production template.

If the system prompt is dynamic, keep that same structure in training with representative values.

PYTHON

```
def build_training_example(user_input, target_output) → dict:
    """Generate data with the SAME builder production uses."""
    return {"messages": [
        {"role": "system", "content": SYSTEM_PROMPT}, # the real one
        {"role": "user", "content": user_input},
        {"role": "assistant", "content": target_output},
    ]}
```



One source of truth for the prompt. **Import it, never retype it.**



AI Coach John

Follow



 Repost

Four Shapes, Four Purposes

Pick by what you are teaching, not by habit

JSONL

1. Single turn instruction, the most common

```
{"messages": [system, user, assistant]}
```

2. Multi turn, teaches context tracking

```
{"messages": [system,
  {"role": "user", "content": "What is the refund window?"},
  {"role": "assistant", "content": "30 days for physical goods."},
  {"role": "user", "content": "What about digital?"},
  {"role": "assistant", "content": "Non-refundable once accessed."}]}
```

3. Tool calling, teaches when and how to call

```
{"messages": [user,
  {"role": "assistant", "tool_calls": [{"function": {
    "name": "get_order", "arguments": {"id": "4471"}}}],
  {"role": "tool", "content": {"status": "shipped"}},
  {"role": "assistant", "content": "Order 4471 has shipped."}]}
```

4. Preference pairs, for DPO style alignment, not SFT

```
{"prompt": ..., "chosen": ..., "rejected": ... }
```

Fixed input to fixed output shape

SINGLE TURN

Conversational assistant

MULTI TURN

Agent that calls tools

TOOL FORMAT

Aligning to subjective preference

DPO PAIRS



Most trainers compute loss on **every** assistant turn, so every assistant message in a multi turn example must be one you would be happy for the model to imitate.



AI Coach John

Follow



Repost

Train On Completions Only

The model should learn to write answers, not questions

PYTHON

```
def build_labels(tokenizer, prompt_text, completion_text):  
    """Mask the prompt so loss lands on the completion."""  
    prompt_ids = tokenizer(prompt_text,  
                            add_special_tokens=False).input_ids  
    full_ids = tokenizer(prompt_text + completion_text,  
                        add_special_tokens=False).input_ids  
  
    labels = list(full_ids)  
    labels[: len(prompt_ids)] = [-100] * len(prompt_ids) # ignored  
    return {"input_ids": full_ids, "labels": labels}
```

✗ WITHOUT MASKING

system: train on it
user: train on it
assistant: train on it

Capacity spent learning to write user messages, which can bleed into outputs

✓ WITH MASKING

system: ignored
user: ignored
assistant: TRAIN ON THIS

All gradient goes to the behaviour you want



Managed APIs and high level libraries handle this. **Writing your own loop makes it your job, and forgetting it is a quiet quality tax.**



Fewer Than You Think, If They Are Consistent

Indicative ranges, always validated on your own task

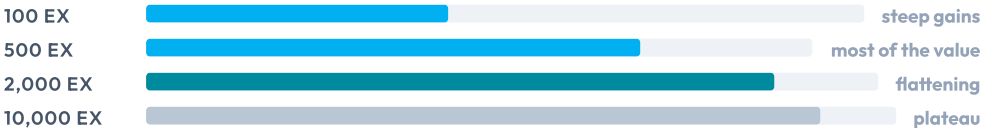
50 to 200 Narrow format enforcement, always return this JSON schema

200 to 1k Tone and style adaptation, write like our support team

1k to 5k Domain task with reasoning, classify and explain in our taxonomy

10k plus Broad capability change, usually better solved another way

THE SHAPE OF THE RETURNS CURVE



500 carefully checked examples beat 5,000 unchecked ones. Past a few thousand, more of the same moves nothing. Different things do.



AI Coach John

Follow

PROFITBRIDGE

Repost

One Odd Example Teaches An Exception

The top quality dimension, and the easiest to check

✳ A DATASET WITH A CONSISTENCY BUG

```
ex 1 {"category": "billing", "severity": "high"}
ex 2 {"category": "billing", "severity": "high"}
ex 3 {"category": "Billing", "severity": "High"}    < capitalisation
ex 4 {"severity": "high", "category": "billing"}    < key order
ex 5 {"category": "billing", "severity": 3}          < type change
ex 6 Category: billing, Severity: high              < not JSON
```

The model does not conclude "those were mistakes". It concludes "sometimes the format differs" and reproduces that in production.

PYTHON

```
def check_output_consistency(examples) → dict:
    shapes, failures = Counter(), []
    for i, ex in enumerate(examples):
        out = ex["messages"][-1]["content"]
        try:
            shapes[tuple(sorted(json.loads(out).keys()))] += 1
        except json.JSONDecodeError:
            failures.append(i)
    return {"distinct_key_sets": dict(shapes), "non_json": failures}

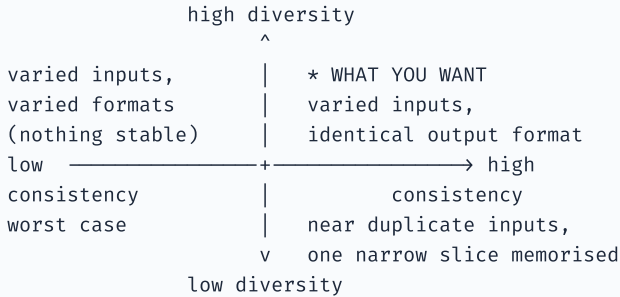
# Healthy output: ONE key set, zero failures.
```



Same Shape, Varied Substance

You need diversity and consistency at the same time

WHERE YOU WANT TO LAND



✂ VARY THESE

input phrasing and length
vocabulary and register
edge cases and ambiguity
topics and categories
typos, fragments, real mess
difficulty

🔒 NEVER VARY THESE

output schema and structure
field names and ordering
tone and register of answers
refusal phrasing
citation format
the system prompt



Four Sources, Ranked

In descending order of what each one is worth

-
- 01 Human corrected production outputs.** Real input, the model's attempt, a human's fix. Every correction is a labelled example of the exact failure you want gone. **BEST**
-
- 02 Real inputs, expert written outputs.** The real input distribution with high quality targets. Slower to produce, still very valuable. **HIGH**
-
- 03 Hand written seeds.** 50 to 100 examples from someone who knows the task. Essential as a style anchor even if you scale synthetically. **ANCHOR**
-
- 04 Synthetic expansion.** Generated from those seeds, filtered and sampled for verification. Never the only source. **CAREFUL**
-

↗ THE PATTERN THAT WORKS

Hand write a small seed set, expand it synthetically, then progressively replace synthetic examples with human corrected production data as it accumulates.

∞ Done this way, **the dataset improves as the system runs** instead of ageing the moment you ship.



Your Failures Are Your Best Examples

Prioritise the interactions that reveal a gap

PYTHON

```
def mine_training_candidates(logs) → list[dict]:
    candidates = []
    for e in logs:
        # Highest value: a human corrected the output
        if e.get("human_correction"):
            prio, target = "corrected", e["human_correction"]
        # High: negative feedback with a known good answer
        elif e.get("feedback") == "negative" and e.get("resolved"):
            prio, target = "negative", e["resolved"]
        # Failed validation needs a human to write the target
        elif e.get("validation_errors"):
            continue
        # Low: positive runs reinforce existing behaviour
        elif e.get("feedback") == "positive":
            prio, target = "positive", e["output"]
        else:
            continue
    candidates.append({"input": e["input"], "target": target,
                      "priority": prio, "req_id": e["request_id"]})
    return candidates
```

★ WHY CORRECTIONS BEAT EVERYTHING ELSE

A correction encodes three things at once: a real input from the real distribution, the model's actual failure on it, and the exact target a human considers correct. Synthetic data can invent plausible inputs. It cannot tell you where your model actually fails.



Anonymise before training. Production logs carry PII, and fine-tuning bakes whatever you show it into the weights permanently.



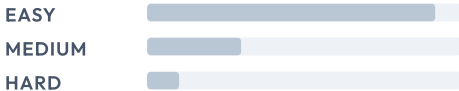
Train On What It Currently Gets Wrong

Examples it already handles teach nothing

PYTHON

```
def find_capability_gaps(candidates, base_model) → list[dict]:  
    """Keep the examples where the base model fails today."""  
    gaps = []  
    for c in candidates:  
        prediction = base_model.generate(c["input"])  
        if not matches(prediction, c["target"]):  
            gaps.append({**c, "base_model_output": prediction})  
    return gaps
```

× TYPICAL BUILD



Most of the signal teaches what it knows

✓ TARGETED BUILD



Gradient spent where capability is missing



Do not remove easy examples entirely. Keep a meaningful minority, or behaviour on the common cases can shift too.



AI Coach John

Follow



Repost

Teach It What Not To Answer

Refusals and deferrals belong in the dataset

JSONL

Out of scope, teaches the boundary of the task

```
{"messages": [system, {"role": "user", "content":  
  "What do you think about the election?"},  
  {"role": "assistant", "content": "I can only help with your  
  account and orders. Is there something I can look up?"}]}
```

Insufficient information, teaches honest uncertainty

```
{"messages": [system, {"role": "user", "content":  
  "When will my order arrive?"},  
  {"role": "assistant", "content": "I would need your order  
  number to check. Could you share it?"}]}
```

Missing knowledge, teaches deferral instead of invention

```
{"messages": [system, {"role": "user", "content":  
  "What is the refund policy in Japan?"},  
  {"role": "assistant", "content": "I do not have region  
  specific policy for Japan. I can connect you with support."}]}
```

! WITHOUT THESE

A thousand examples where the assistant always produces a confident, complete answer teach exactly one thing: **always produce a confident, complete answer**. It then answers out of scope questions confidently and invents details when information is missing.

= Keep refusal phrasing **identical across every refusal example**. Varied wording produces unpredictable refusals, the one place you least want variance.



Cheaper Than A Wasted Training Run

Structural, content, length and duplicate checks

PYTHON

```
def validate_dataset(examples, tokenizer, max_tokens) → dict:
    problems, lengths, seen = Counter(), [], set()
    for ex in examples:
        msgs = ex.get("messages", [])
        # Structural
        if not msgs or msgs[-1]["role"] ≠ "assistant":
            problems["malformed"] += 1; continue
        # Content
        target = (msgs[-1]["content"] or "").strip()
        if len(target) < 5: problems["trivial_target"] += 1
        # Length, truncation silently destroys examples
        n = len(tokenizer.apply_chat_template(msgs))
        lengths.append(n)
        if n > max_tokens: problems["exceeds_context"] += 1
        # Duplicates
        key = hashlib.sha256(json.dumps(msgs, sort_keys=True).encode())
        if key.hexdigest() in seen: problems["duplicate"] += 1
        seen.add(key.hexdigest())
    return {"problems": dict(problems), "n": len(examples),
            "p95": int(np.percentile(lengths, 95)), "max": max(lengths)}
```

THE LENGTH CHECK PEOPLE SKIP

```
max_tokens = 4096      token_p95 = 3,890    fine
                        token_max = 11,204  < 40 examples over
```

Those 40 get truncated during training. The assistant response, the only part carrying loss, sits at the end. **Truncation removes it entirely**, and nothing in the logs will tell you.



Hold Out A Real Test Set First

Split by source, never by example

PYTHON

```
def split_dataset(examples, val=0.1, test=0.1, seed=42):
    """Split by SOURCE so related items cannot leak."""
    by_source = defaultdict(list)
    for ex in examples:
        by_source[ex["meta"]["source_id"]].append(ex)

    sources = sorted(by_source)
    random.Random(seed).shuffle(sources)
    n_test, n_val = int(len(sources) * test), int(len(sources) * val)
    test_src = set(sources[:n_test])
    val_src = set(sources[n_test:n_test + n_val])

    held = test_src | val_src
    train = [e for s, g in by_source.items() if s not in held for e in g]
    return train, [e for s in val_src for e in by_source[s]], \
        [e for s in test_src for e in by_source[s]]
```

TRAIN

The model sees these and updates weights on them

VAL

Watched during training. Validation loss rising while training loss falls means you are memorising, so stop.

TEST

Touched once, at the very end. Tune anything on it and it becomes another validation set.



Two examples from the same source are not independent. Splitting them across the boundary inflates every number you report.



Before, During And After

How you know the dataset is doing its job

BEFORE

- ✓ One output shape
- ✓ Nothing over the context limit
- ✓ Duplicate rate low
- ✓ 30 to 50 read by a human
- ✓ Includes cases the base fails
- ✓ Refusals present

DURING

- ✓ Training loss falls smoothly, not to zero
- ✓ Validation loss falls then flattens
- ✓ Rising val loss means overfit, cut epochs
- ✓ Checkpoint samples look like your targets

AFTER

- ✓ Format adherence on held out data
- ✓ Accuracy vs the base model, same test set
- ✓ Regression check outside the domain
- ✓ Refusals still fire on out of scope input



The regression check is the one people skip. Narrow fine-tuning can measurably degrade capability outside the target domain, so test general behaviour before and after.



What Wastes A Training Run

Every one of these is a dataset problem

× BUILDING IT

fine-tuning to inject facts
training format differs from production
inconsistent output format
near duplicates inflating the count
only easy examples
no refusal or out of scope examples
varied refusal wording

× RUNNING IT

examples over the context limit
splitting train and test by example
tuning against the test set
training on unanonymised logs
no regression check afterwards
chasing more data instead of checking it

SYMPTOM	LIKELY CAUSE
"format is inconsistent"	inconsistent training examples
"worse than the base model"	wrong task for fine-tuning
"great on test, poor in prod"	prompt format mismatch
"makes things up more now"	no refusal examples in the data
"eval suspiciously high"	train and test leakage
"no improvement at all"	trained on cases it already passed
"broke unrelated things"	narrow data, no regression check



Before You Start A Training Run

Six groups, one pass, no compute spent yet

DECISION

- ✓ Prompting and RAG tried, insufficient
- ✓ Goal is behaviour, not knowledge
- ✓ A prompt can produce it at least sometimes

FORMAT

- ✓ Structure matches production exactly
- ✓ Built from the same prompt template
- ✓ One output schema everywhere
- ✓ Loss masked to completions

CONTENT

- ✓ Inputs reflect real user messiness
- ✓ Weighted toward current failures
- ✓ Refusal and edge cases present
- ✓ Refusal phrasing identical

QUALITY

- ✓ Validation script run, zero malformed
- ✓ Zero examples over the length limit
- ✓ Duplicate rate measured and low
- ✓ 30 to 50 read, error rate recorded
- ✓ PII removed from production data

SPLITS

- ✓ Split by source, not by example
- ✓ Test set untouched until the end

AFTER

- ✓ Format adherence measured
- ✓ Compared to base on the same test set
- ✓ Regression check outside the domain



The Dataset Is The Model

Hyperparameters get a few percent. The data decides what it becomes.

DO THIS	AND YOU GET
MATCH PRODUCTION FORMAT	no train and inference gap
ONE OUTPUT SCHEMA	predictable behaviour
VARY THE INPUTS	generalises instead of memorising
TARGET REAL FAILURES	gradient spent where it is needed
INCLUDE REFUSALS	knows what not to answer
CHECK LENGTH LIMITS	no silently truncated targets
SPLIT BY SOURCE	honest evaluation
READ A SAMPLE	the check nothing else replaces
REGRESSION TEST AFTER	catches what narrow tuning broke



It learns the format, the tone and the boundaries you demonstrate, **plus every inconsistency you leave in.** Follow AI Coach John for more.



AI Coach John

[Follow](#)



[Repost](#)